CH-1

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

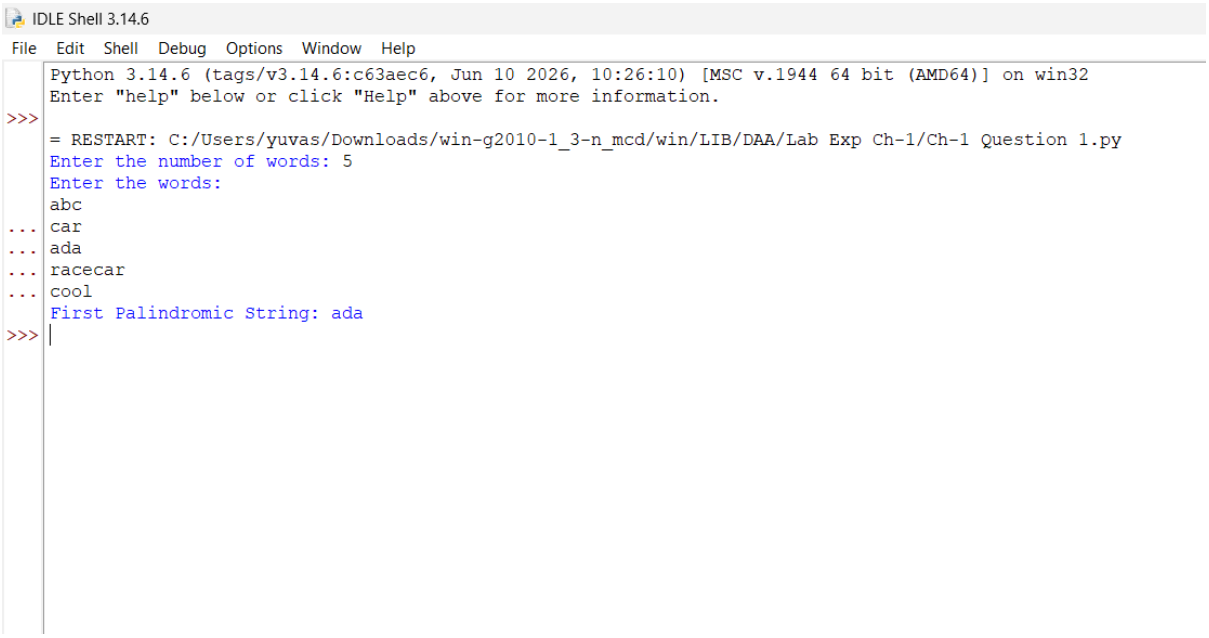
Example 1:

Input:

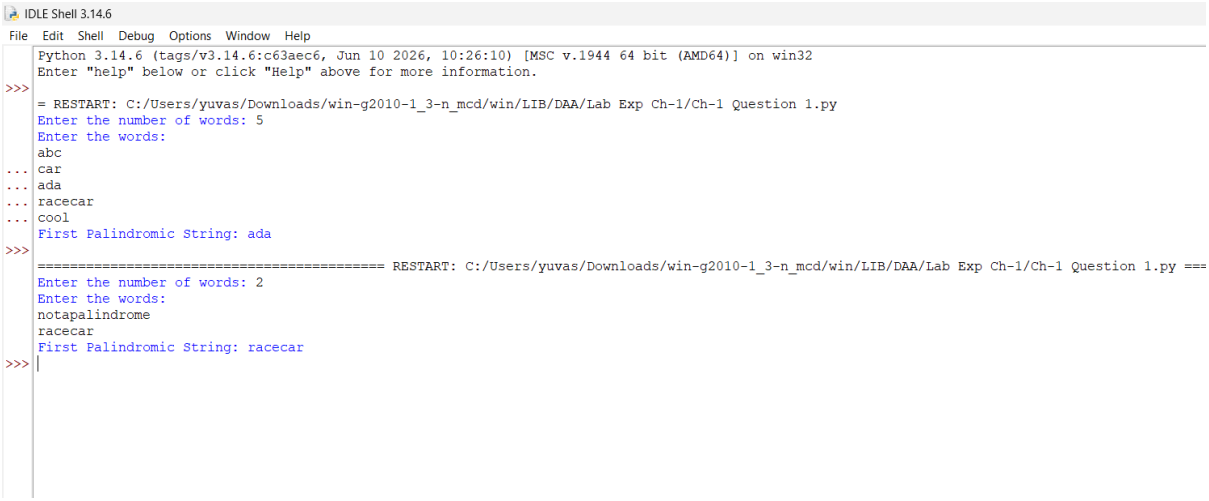
```
def first_palindrome(words):
    for word in words:
        if word == word[::-1]:
            return word
    return ""


n = int(input("Enter the number of words: "))


words = []


print("Enter the words:")
for i in range(n):
    words.append(input())


result = first_palindrome(words)


print("First Palindromic String:", result)
```

output:

Example 1 :



Example 2:



2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

Example 1:

Input: nums1 = [2,3,2], nums2 = [1,2]

Output: [2,1]

Explanation:

Example 2:

Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6]

Output: [3,4]

Explanation:

The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So answer1 is 3.

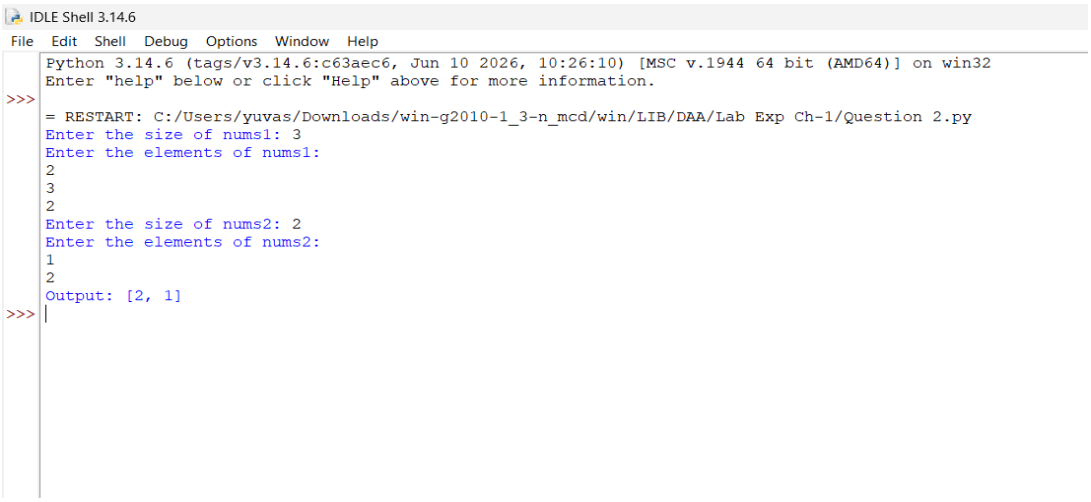
The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.

Input:

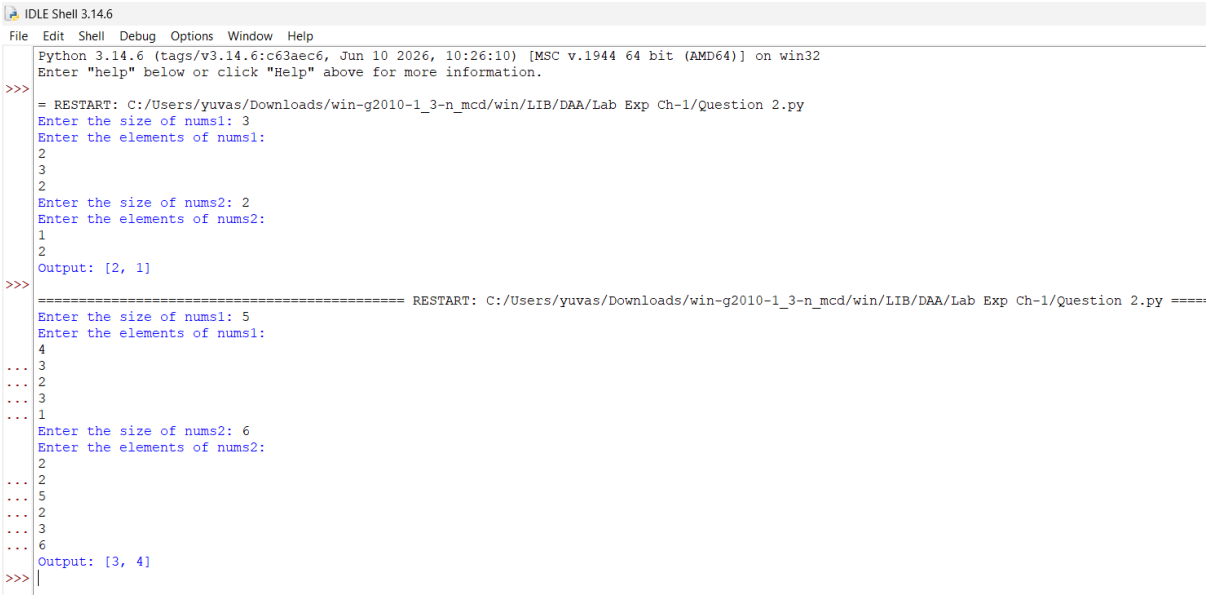
```
def find_intersection_values(nums1, nums2):
    answer1 = 0
    answer2 = 0

    for num in nums1:
        if num in nums2:
            answer1 += 1

    for num in nums2:
        if num in nums1:
            answer2 += 1

    return [answer1, answer2]

n = int(input("Enter the size of nums1: "))
nums1 = []

print("Enter the elements of nums1:")
for i in range(n):
    nums1.append(int(input()))
```

```
m = int(input("Enter the size of nums2: "))
nums2 = []

print("Enter the elements of nums2:")
for i in range(m):
    nums2.append(int(input()))

result = find_intersection_values(nums1, nums2)

print("Output:", result)
```

Example 1 :



Example 2:



3. You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is

defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j

such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is

called the distinct count of nums[i..j]. Return the sum of the squares of distinct counts of

all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within

an array.

Example 1:

Input: nums = [1,2,1]

Output: 15

Explanation: Six possible subarrays are:

[1]: 1 distinct value

[2]: 1 distinct value

[1]: 1 distinct value

[1,2]: 2 distinct values

[2,1]: 2 distinct values

[1,2,1]: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to 12 + 12 + 12 + 22 + 22 + 22 = 15.

Example 2:

Input: nums = [1,1]

Output: 3

Explanation: Three possible subarrays are:

[1]: 1 distinct value

[1]: 1 distinct value

[1,1]: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to 12 + 12 + 12 = 3.

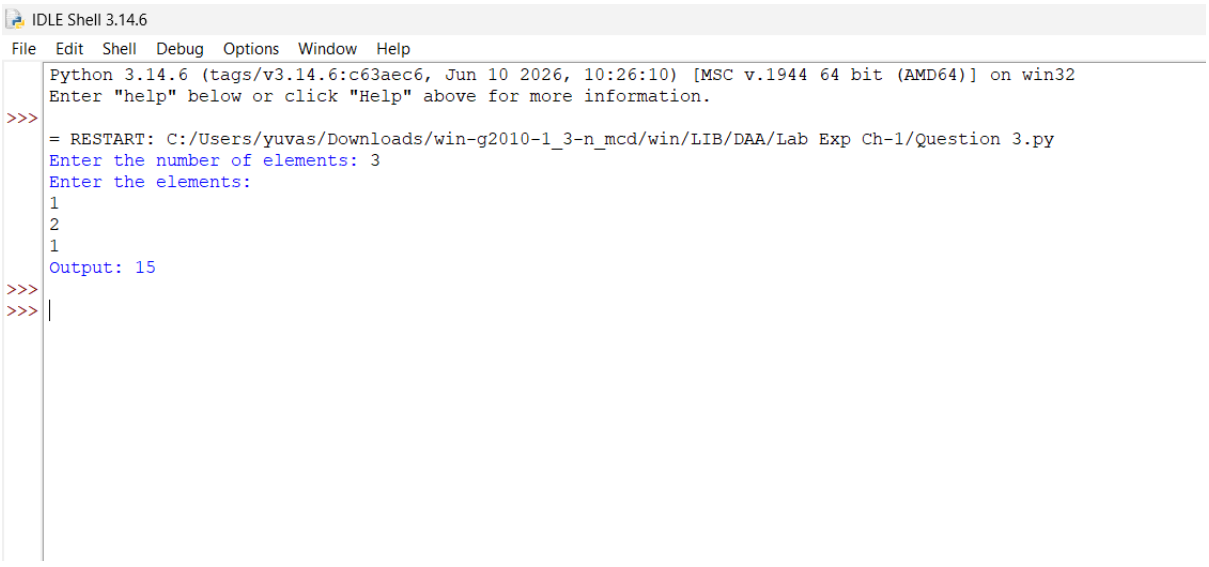
INPUT:

```
def sum_of_squares(nums):
    total = 0
    n = len(nums)

    for i in range(n):
        s = set()

        for j in range(i, n):
            s.add(nums[j])
            distinct = len(s)
            total = total + distinct * distinct

    return total

n = int(input("Enter the number of elements: "))
```

```
nums = []


print("Enter the elements:")
for i in range(n):
    nums.append(int(input()))


answer = sum_of_squares(nums)


print("Output:", answer)
```

OUTPUT

EXAMPLE 1



EXAMPLE 2


IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/yuvas/Downloads/win-g2010-1_3-n_mcd/win/LIB/DAA/Lab Exp Ch-1/Question 3.py
Enter the number of elements: 3
Enter the elements:
1
2
1
Output: 15
>>>
>>>
========== RESTART: C:/Users/yuvas/Downloads/win-g2010-1_3-n_mcd/win/LIB/DAA/Lab Exp Ch-1/Question 3.py ==========
Enter the number of elements: 2
Enter the elements:
1
1
Output: 3
>>>

4. Given a 0-indexed integer array nums of length n and an integer k, return the number of

pairs (i, j) where 0 <= i < j < n, such that nums[i] == nums[j] and (i * j) is divisible by k.

Example 1:

Input: nums = [3,1,2,2,2,1,3], k = 2

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- nums[0] == nums[6], and 0 * 6 == 0, which is divisible by 2.

- nums[2] == nums[3], and 2 * 3 == 6, which is divisible by 2.

- nums[2] == nums[4], and 2 * 4 == 8, which is divisible by 2.

- nums[3] == nums[4], and 3 * 4 == 12, which is divisible by 2.

Example 2:

Input: nums = [1,2,3,4], k = 1

Output: 0

Explanation: Since no value in nums is repeated, there are no pairs (i,j) that meet all the requirements.

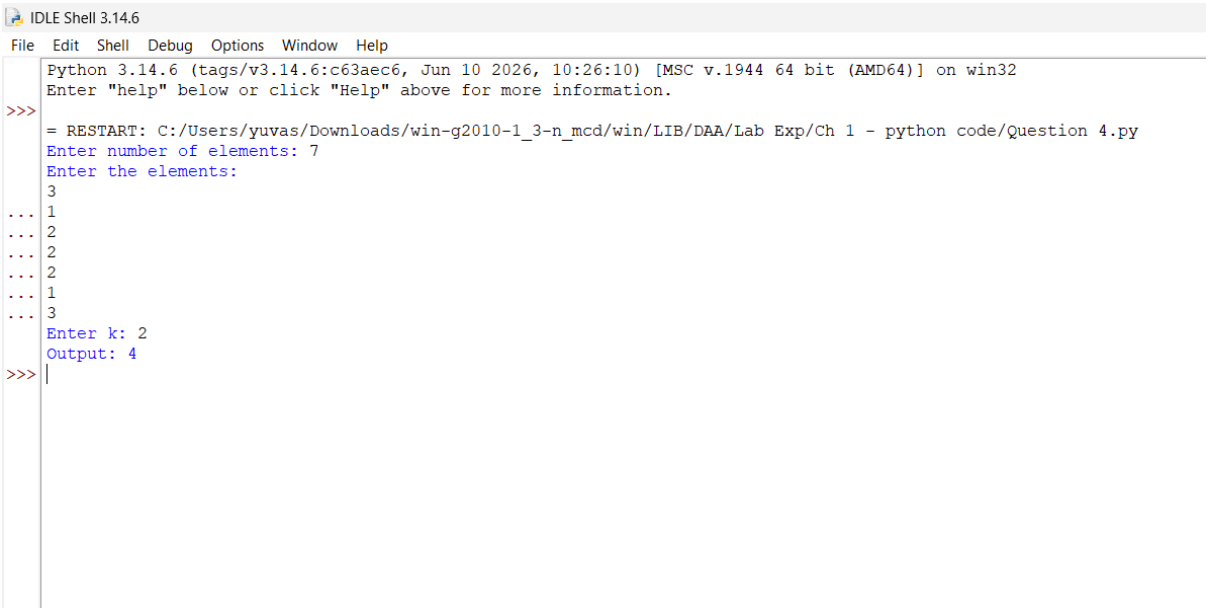
INPUT :

```
def countPairs(nums, k):
    count = 0
    n = len(nums)

    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1

    return count
```

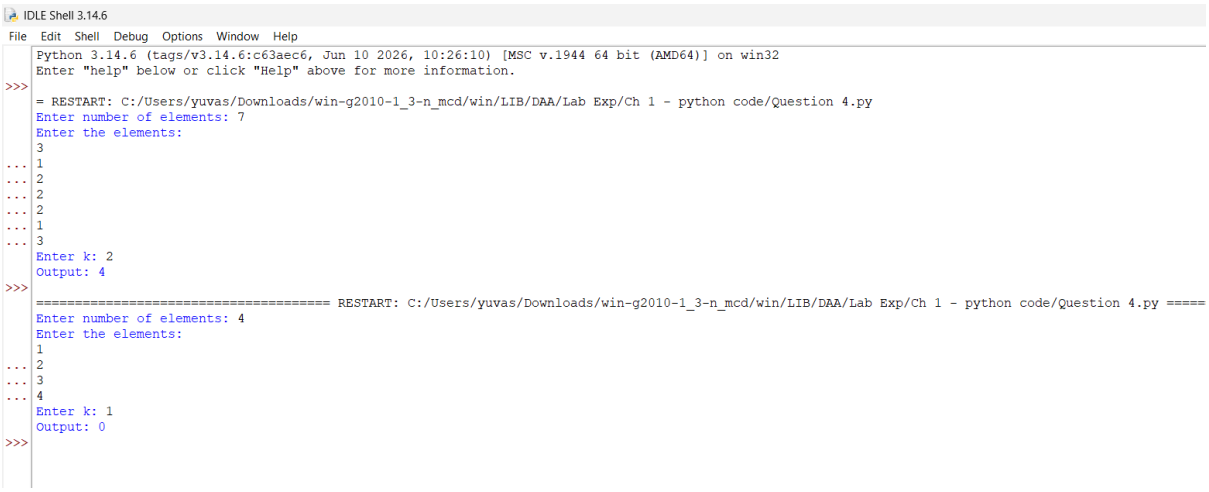
```
n = int(input("Enter number of elements: "))

nums = []
print("Enter the elements:")
for i in range(n):
    nums.append(int(input()))

k = int(input("Enter k: "))

print("Output:", countPairs(nums, k))
```

OUTPUT:

EXAMPLE 1



EXAMPLE 2


IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/yuvas/Downloads/win-g2010-1_3-n_mcd/win/LIB/DAA/Lab Exp/Ch 1 - python code/Question 4.py
Enter number of elements: 7
Enter the elements:
3
1
2
2
2
1
3
Enter k: 2
Output: 4
>>>
========== RESTART: C:/Users/yuvas/Downloads/win-g2010-1_3-n_mcd/win/LIB/DAA/Lab Exp/Ch 1 - python code/Question 4.py =====
Enter number of elements: 4
Enter the elements:
1
2
3
4
Enter k: 1
Output: 0
>>>

5. Write a program FOR THE BELOW TEST CASES with least time complexity

Test Cases: -

1) Input: {1, 2, 3, 4, 5} Expected Output: 5

2) Input: {7, 7, 7, 7, 7} Expected Output: 7

3) Input: {-10, 2, 3, -4, 5} Expected Output: 5

INPUT:

```
def findMax(arr):
    maximum = arr[0]

    for i in arr:
        if i > maximum:
            maximum = i

    return maximum

n = int(input("Enter number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

print("Output:", findMax(arr))
```

OUTPUT:


Python 3.8.2 Shell
File Edit Shell Debug Options Window Help
Python 3.8.2 (tags/v3.8.2:7b3ab59, Feb 25 2020, 23:03:10) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/yuvas/Downloads/win-g2010-1_3-n_mcd/win/LIB/DAA/Lab Exp/Ch 1 - python code/Question 5.py
Enter number of elements: 5
Enter the elements:
1
2
3
4
5
Output: 5
>>>
========== RESTART: C:/Users/yuvas/Downloads/win-g2010-1_3-n_mcd/win/LIB/DAA/Lab Exp/Ch 1 - python code/Question 5.py ==========
Enter number of elements: 5
Enter the elements:
7
7
7
7
7
Output: 7
>>>
========== RESTART: C:/Users/yuvas/Downloads/win-g2010-1_3-n_mcd/win/LIB/DAA/Lab Exp/Ch 1 - python code/Question 5.py ==========
Enter number of elements: 5
Enter the elements:
-10
2
3
-4
5
Output: 5
>>>

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

7. Empty List

8. Input: []

9. Expected Output: None or an appropriate message indicating that the list is empty.

10. Single Element List

11. Input: [5]

12. Expected Output: 5

13. All Elements are the Same

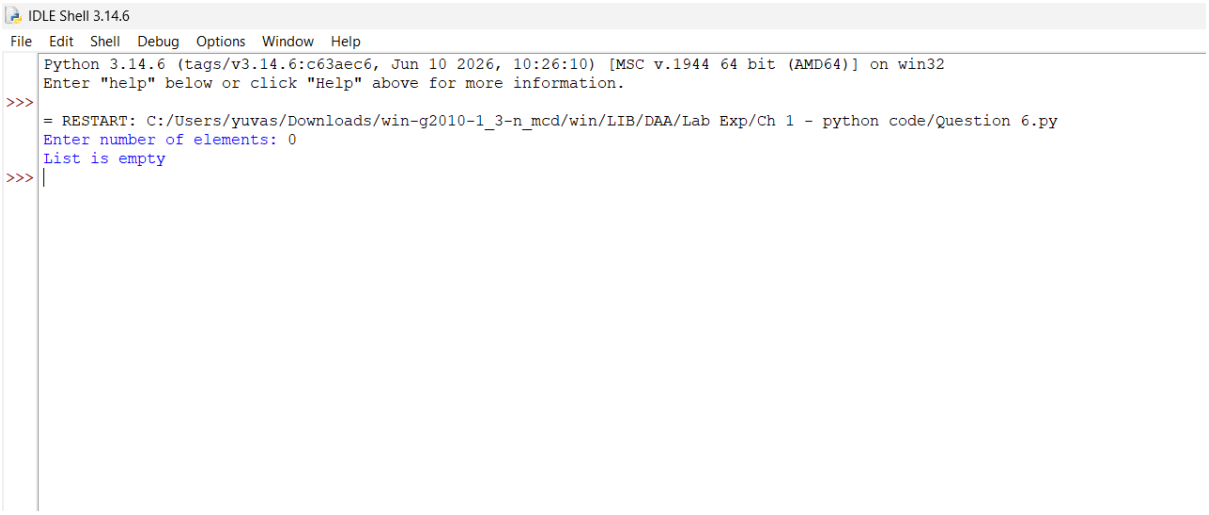
14. Input: [3, 3, 3, 3, 3]

15. Expected Output: 3

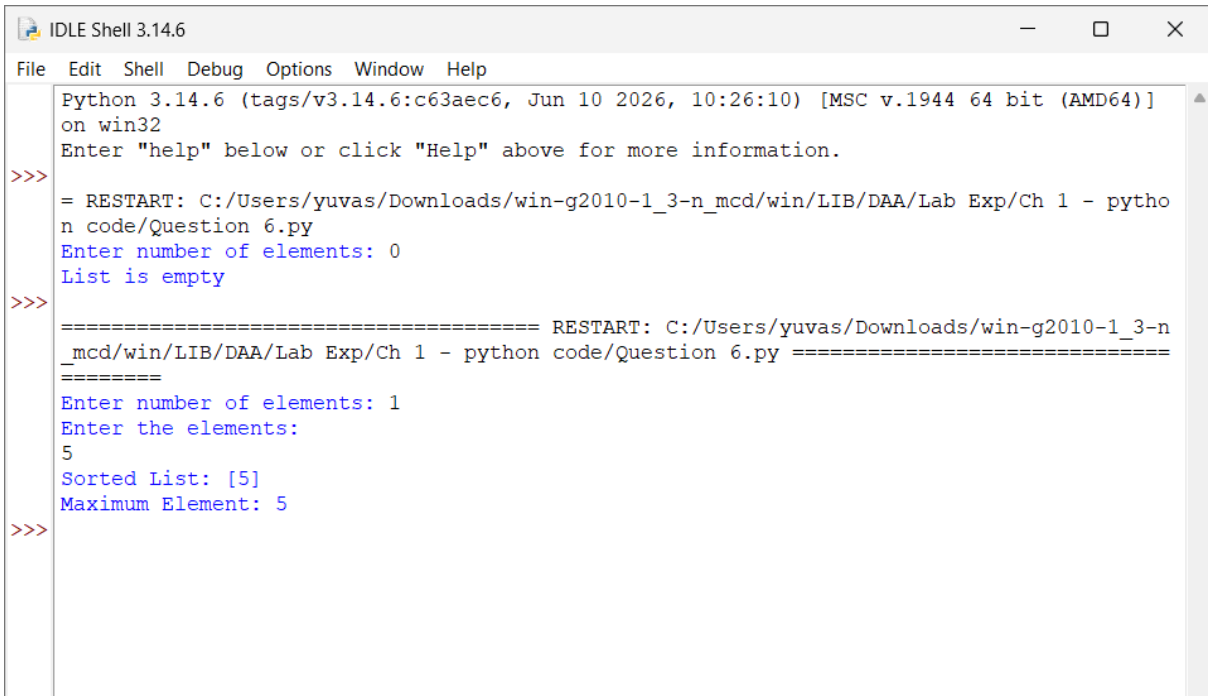
PYTHON CODE:

```
n = int(input("Enter number of elements: "))


arr = []


if n == 0:
    print("List is empty")
else:
    print("Enter the elements:")
    for i in range(n):
        arr.append(int(input()))


    arr.sort()


    print("Sorted List:", arr)
    print("Maximum Element:", arr[-1])
```

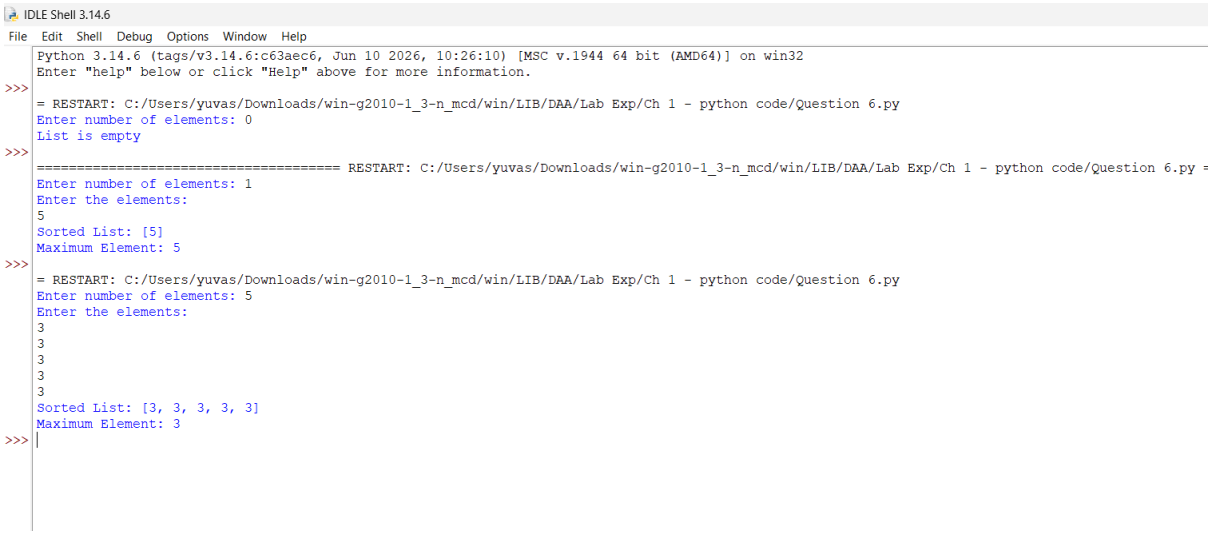
Sample Output 1 (Empty List)



Sample Output 2 (Single Element)



Sample Output 3 (All Elements Same)



7. Write a program that takes an input list of n numbers and creates a new list containing

only the unique elements from the original list. What is the space complexity of the

algorithm?

Test Cases

Some Duplicate Elements

 Input: [3, 7, 3, 5, 2, 5, 9, 2]

 Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

 Input: [-1, 2, -1, 3, 2, -2]

 Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

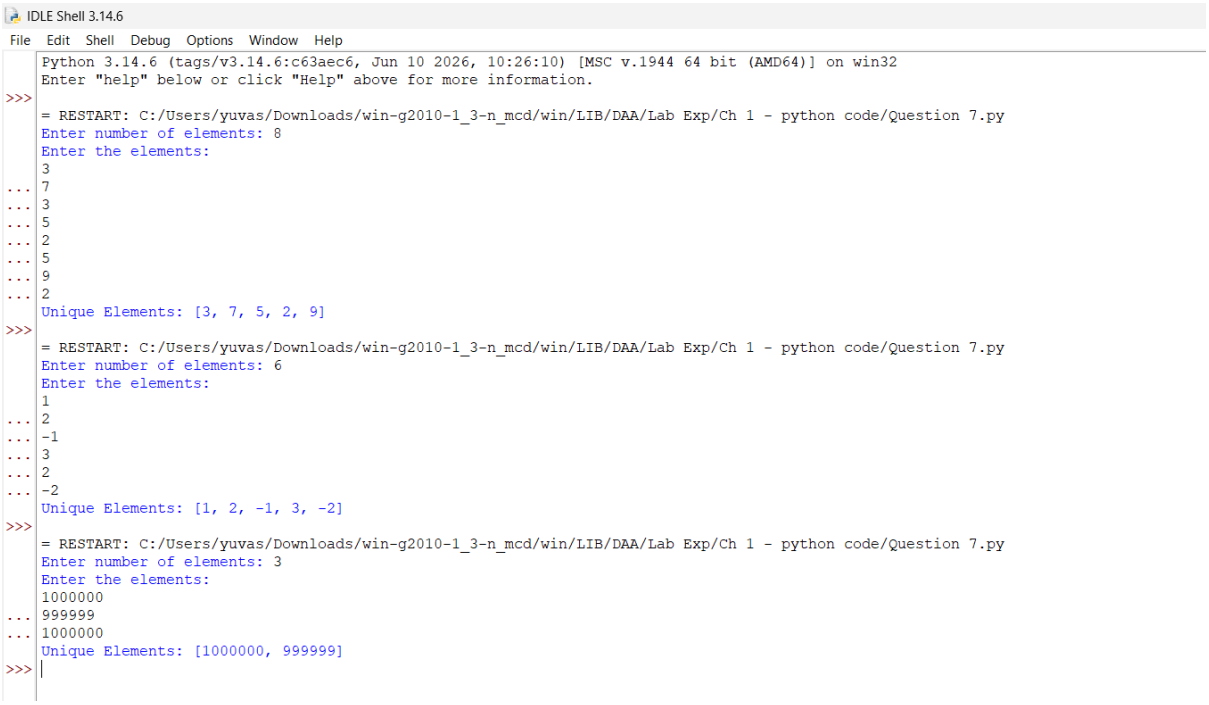
 Input: [1000000, 999999, 1000000]

 Expected Output: [1000000, 999999]

PYTHON CODE:

```
n = int(input("Enter number of elements: "))

arr = []
unique = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

for i in arr:
    if i not in unique:
        unique.append(i)

print("Unique Elements:", unique)
```

Sample Output (1,2,3)

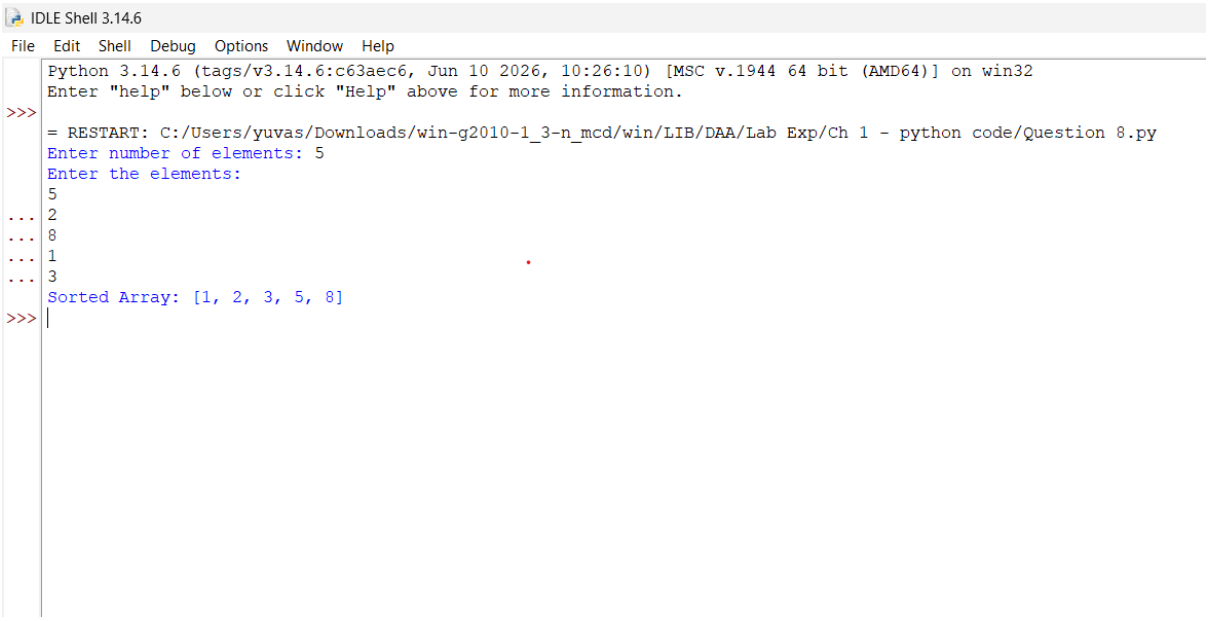


8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

INPUT:

```
n = int(input("Enter number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

for i in range(n - 1):
    for j in range(n - 1 - i):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted Array:", arr)
```

Sample Output



9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its

time complexity using Big-O notation.

Test Case:

Example X={ 3,4,6,-9,10,8,9,30} KEY=10

Output: Element 10 is found at position 5

Example X={ 3,4,6,-9,10,8,9,30} KEY=100

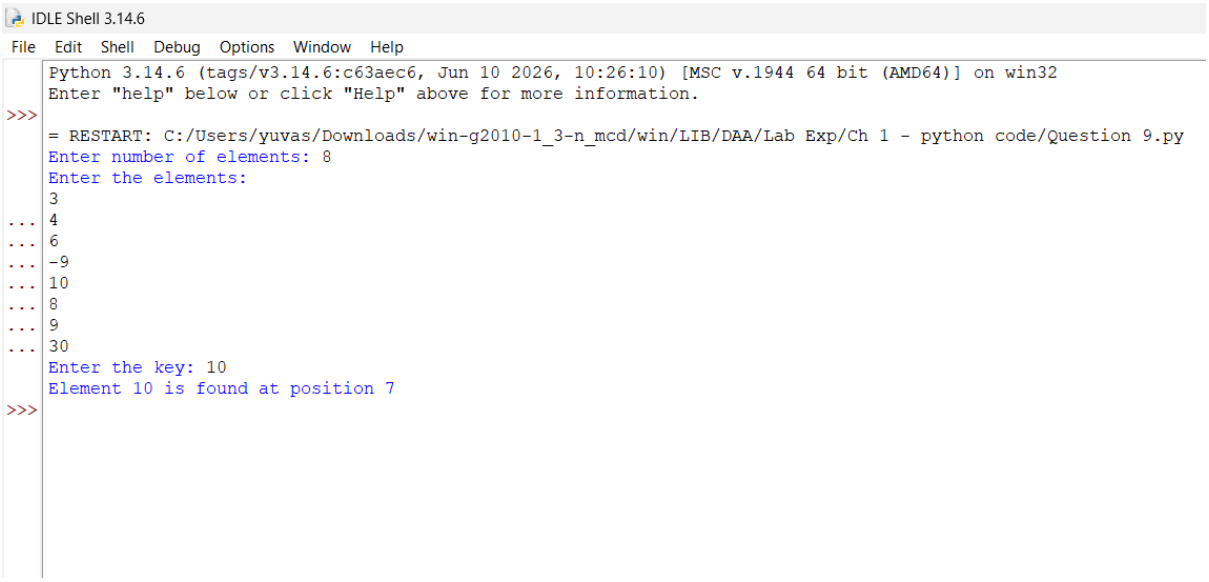
Output : Element 100 is not found

INPUT:

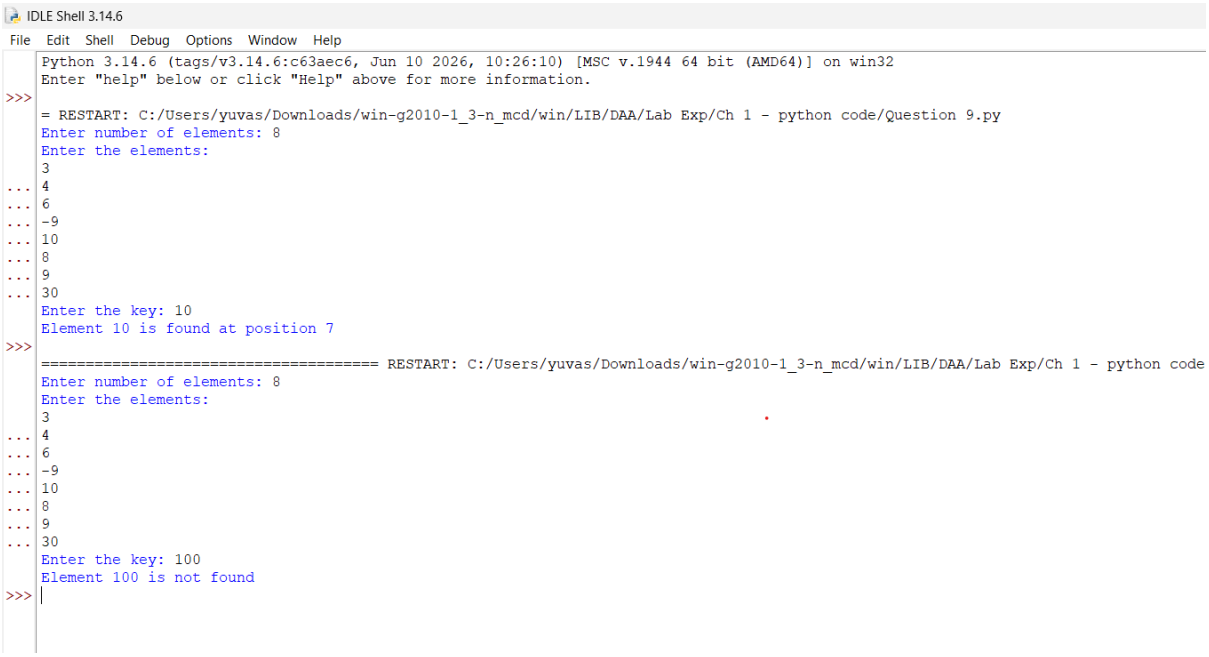
```
n = int(input("Enter number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

key = int(input("Enter the key: "))

arr.sort()

low = 0
high = n - 1
found = False

while low <= high:
    mid = (low + high) // 2
```

```
    if arr[mid] == key:
        print("Element", key, "is found at position", mid + 1)
        found = True
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1

if not found:
    print("Element", key, "is not found")
```

Sample Output 1



Sample Output 2



10. Given an array of integers nums, sort the array in ascending order and return it. You

must solve the problem without using any built-in functions in O(nlog(n)) time

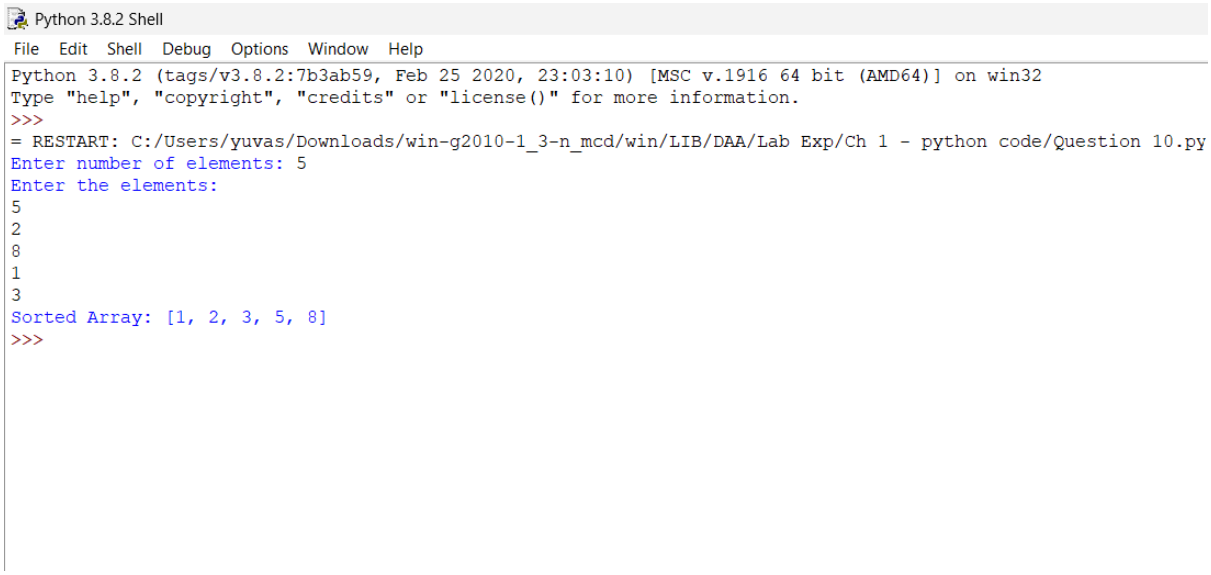
complexity and with the smallest space complexity possible.

INPUT:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2

        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
```

```
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

n = int(input("Enter number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

merge_sort(arr)

print("Sorted Array:", arr)
```

Sample Output



11. Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball

out of the grid boundary in exactly N steps.

Example:

· Input: m=2,n=2,N=2,i=0,j=0 · Output: 6

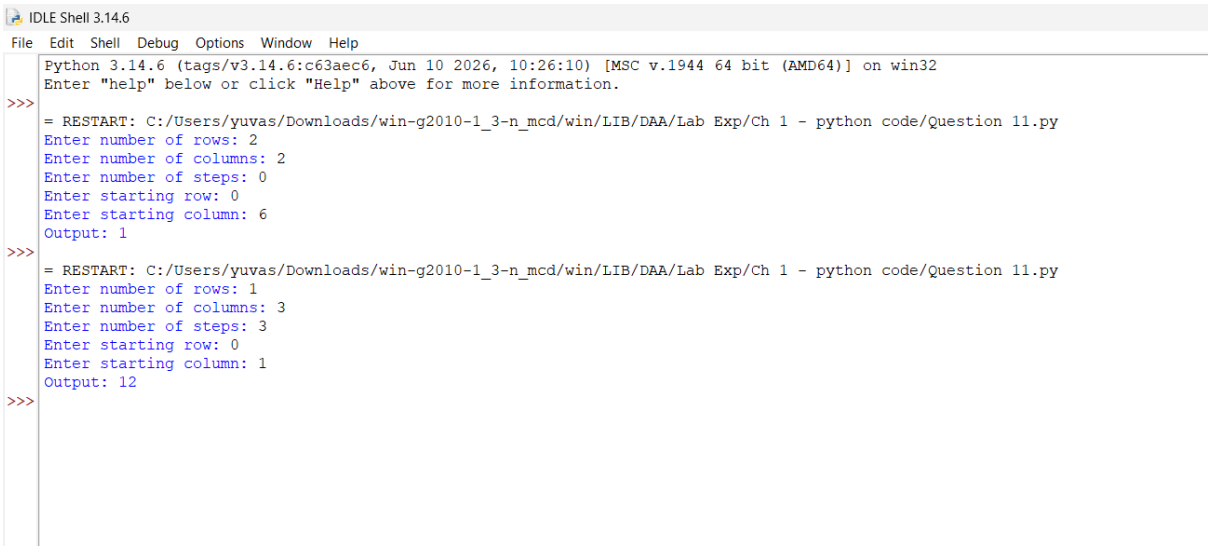
· Input: m=1,n=3,N=3,i=0,j=1 · Output: 12

PYTHON CODE:

```
def findPaths(m, n, N, i, j):
    memo = {}

    def dfs(x, y, steps):
        if x < 0 or x >= m or y < 0 or y >= n:
            return 1
        if steps == 0:
            return 0

        if (x, y, steps) in memo:
            return memo[(x, y, steps)]

        ways = (dfs(x + 1, y, steps - 1) +
                dfs(x - 1, y, steps - 1) +
                dfs(x, y + 1, steps - 1) +
                dfs(x, y - 1, steps - 1))

        memo[(x, y, steps)] = ways
        return ways

    return dfs(i, j, N)
```

```
m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))
N = int(input("Enter number of steps: "))
i = int(input("Enter starting row: "))
j = int(input("Enter starting column: "))

print("Output:", findPaths(m, n, N, i, j))
```

[Sample Output 1] [Sample Output 2]



12.You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

(i) Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3(robbing house 1).

(ii) Input : nums = [1, 2, 3, 1]

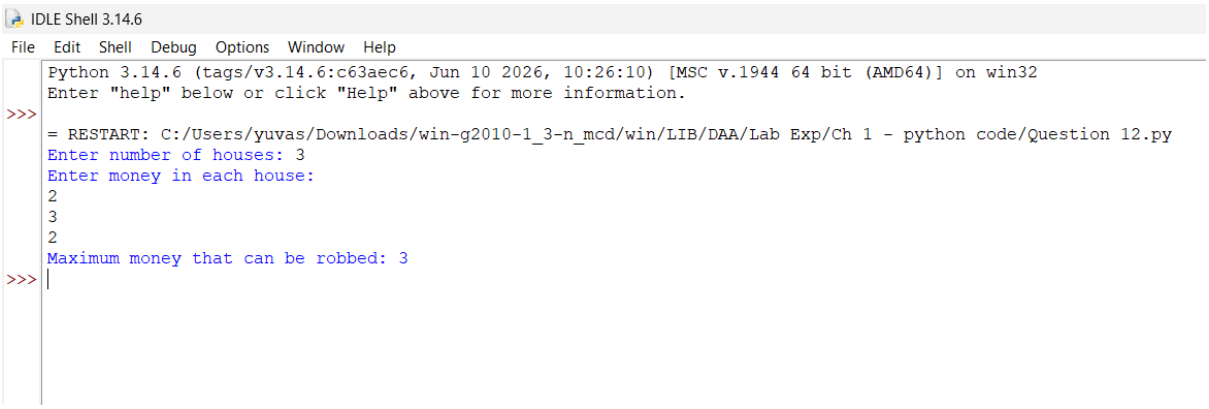
Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

PYTHON CODE:

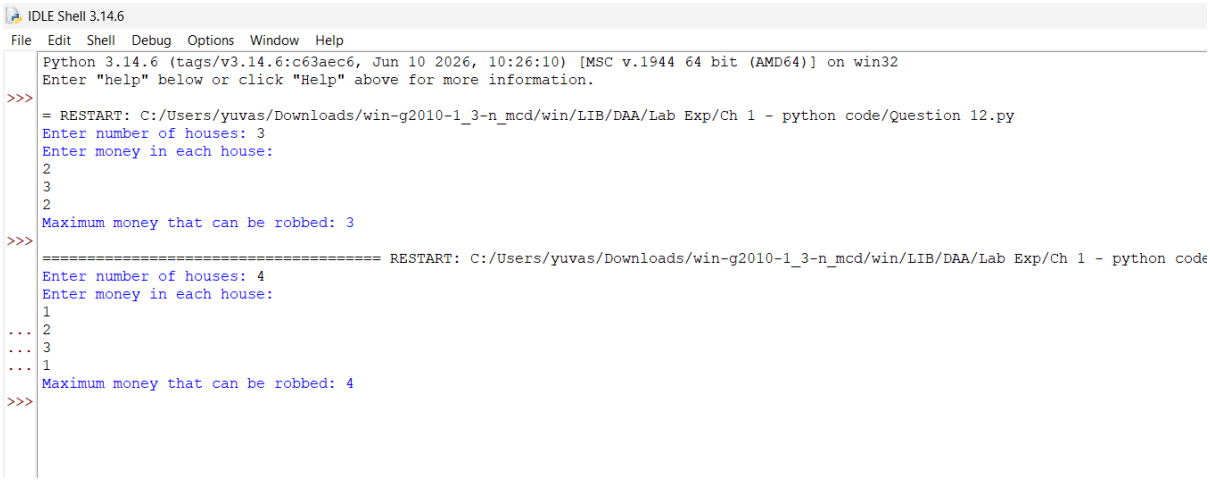
```
def rob_linear(nums):
    prev1 = 0
    prev2 = 0

    for money in nums:
        temp = max(prev1, prev2 + money)
        prev2 = prev1
        prev1 = temp

    return prev1

def rob(nums):
    if len(nums) == 1:
        return nums[0]

    return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))
```

```
n = int(input("Enter number of houses: "))

nums = []

print("Enter money in each house:")
for i in range(n):
    nums.append(int(input()))

print("Maximum money that can be robbed:", rob(nums))
```

Sample Output 1



Sample Output 2



13. You are climbing a staircase. It takes n steps to reach the top. Each time you can either

climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

(i) Input: n=4 Output: 5

(ii) Input: n=3 Output: 3

PYTHON CODE:

```
def climbStairs(n):
    if n <= 2:
        return n

    a = 1
    b = 2

    for i in range(3, n + 1):
        c = a + b
        a = b
        b = c

    return b

n = int(input("Enter the number of steps: "))

print("Number of ways:", climbStairs(n))
```

SAMPLE OUTPUT (1,2)

14. A robot is located at the top-left corner of a m×n grid .The robot can only move either

down or right at any point in time. The robot is trying to reach the bottom-right corner of

the grid. How many possible unique paths are there?

Examples:

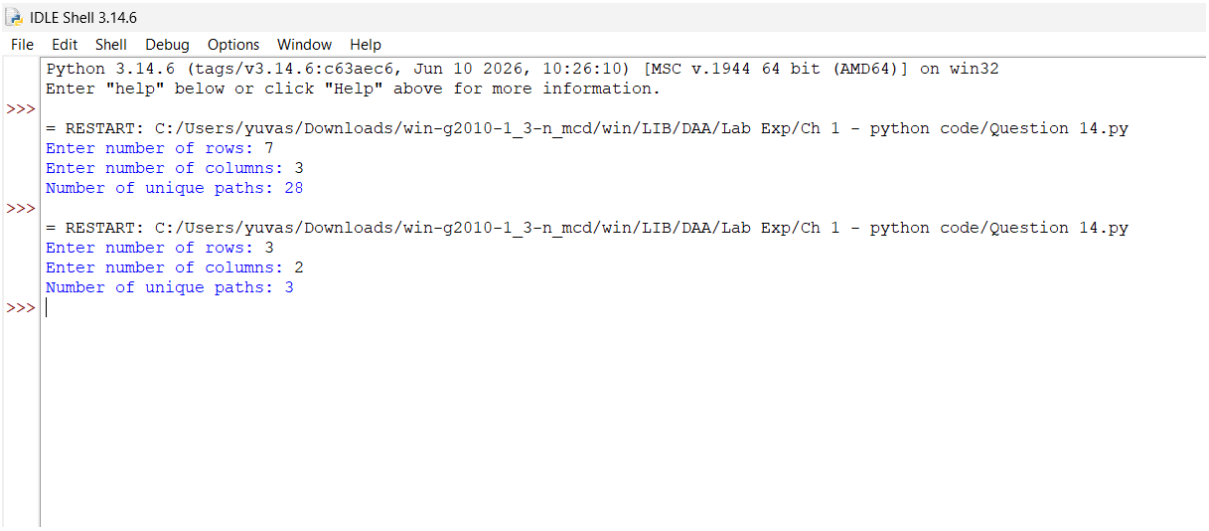
(i) Input: m=7,n=3 Output: 28

(ii) Input: m=3,n=2 Output: 3

PYTHON CODE:

```
def uniquePaths(m, n):
    dp = [[1 for j in range(n)] for i in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]

m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))

print("Number of unique paths:", uniquePaths(m, n))
```

SAMPLE OUTPUT (1,2):



15. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

Example 1:

Input: s = "abbxxxxzzy"

Output: [[3,6]]

Explanation: "xxxx" is the only large group with start index 3 and end index 6.

Example 2:

Input: s = "abc"

Output: []

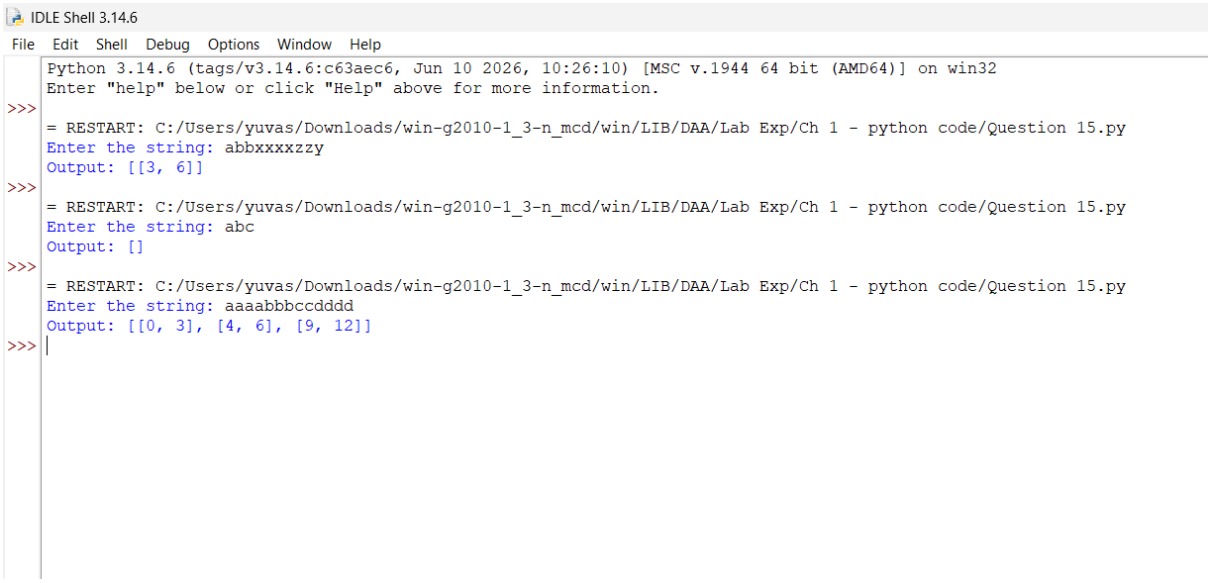
Explanation: We have groups "a", "b", and "c", none of which are large groups.

PYTHON CODE:

```
def largeGroupPositions(s):
    result = []
    n = len(s)
    i = 0

    while i < n:
        start = i

        while i < n - 1 and s[i] == s[i + 1]:
            i += 1
```

```
        if i - start + 1 >= 3:
            result.append([start, i])

        i += 1

    return result

s = input("Enter the string: ")

print("Output:", largeGroupPositions(s))
```

Sample Output (1,2,3)



16. "The Game of Life, also known simply as Life, is a cellular automaton devised by the

British mathematician John Horton Conway in 1970." The board is made up of an m x n

grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

Any live cell with fewer than two live neighbors dies as if caused by under-population.

1. Any live cell with two or three live neighbors lives on to the next generation.
2. Any live cell with more than three live neighbors dies, as if by over-population.
3. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the m x n grid board, return the next state.

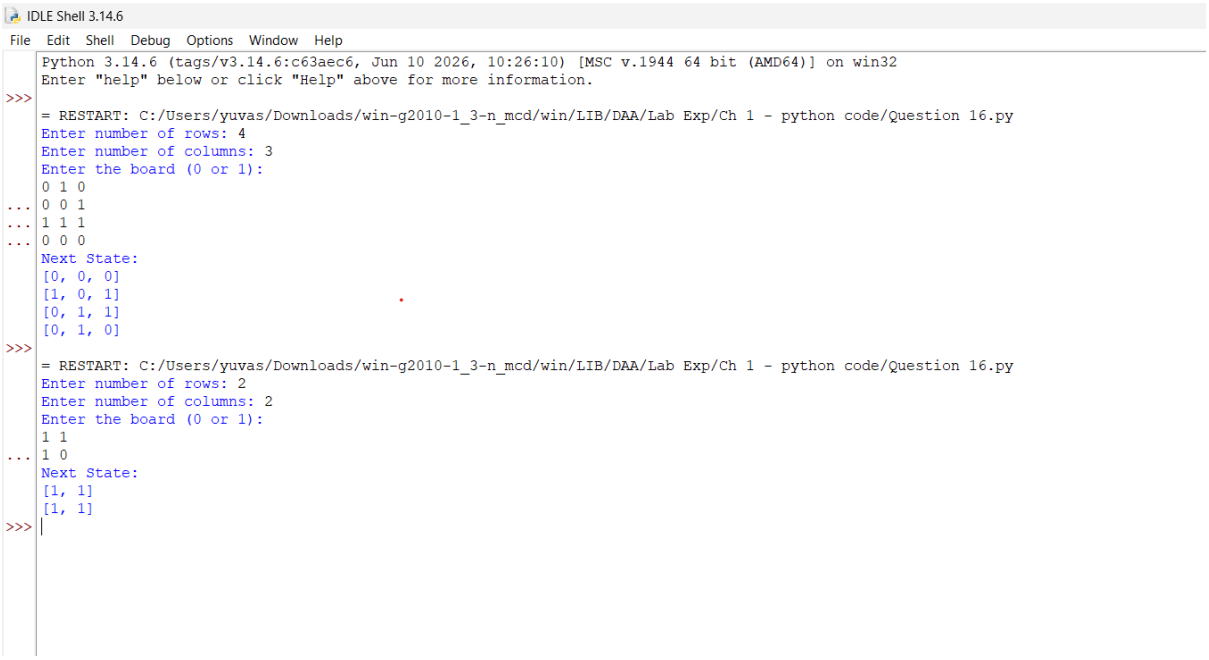
PYTHON CODE:

```
def gameOfLife(board):
    m = len(board)
    n = len(board[0])

    temp = [[0] * n for i in range(m)]

    for i in range(m):
        for j in range(n):
            live = 0
```

```
            for x in [-1, 0, 1]:
                for y in [-1, 0, 1]:
                    if x == 0 and y == 0:
                        continue

                    r = i + x
                    c = j + y

                    if 0 <= r < m and 0 <= c < n:
                        live += board[r][c]

            if board[i][j] == 1:
                if live == 2 or live == 3:
                    temp[i][j] = 1
                else:
                    temp[i][j] = 0
            else:
                if live == 3:
                    temp[i][j] = 1

    return temp

m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))

board = []

print("Enter the board (0 or 1):")
for i in range(m):
```

```
    row = list(map(int, input().split()))
    board.append(row)

result = gameOfLife(board)

print("Next State:")
for row in result:
    print(row)
```

SAMPLE OUTPUT(1,2)



17. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100 th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below\.Now after pouring some non-negative integer cups of champagne, return how full the j th glass in the i th row is (both i and j are 0-indexed.)

Example 1:

Input: poured = 1, query_row = 1, query_glass = 1

Output: 0.00000

Explanation: We poured 1 cup of champange to the top glass of the tower (which

is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top

glass will remain empty.

Example 2:

Input: poured = 2, query_row = 1, query_glass = 1

Output: 0.50000

Explanation: We poured 2 cups of champange to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champange.

PYTHON CODE:

```
def champagneTower(poured, query_row, query_glass):
    dp = [[0.0] * (query_row + 2) for i in range(query_row + 2)]

    dp[0][0] = poured

    for i in range(query_row + 1):
        for j in range(i + 1):
            if dp[i][j] > 1:
                extra = (dp[i][j] - 1) / 2
                dp[i + 1][j] += extra
                dp[i + 1][j + 1] += extra
                dp[i][j] = 1

    return min(1, dp[query_row][query_glass])

poured = int(input("Enter poured cups: "))
query_row = int(input("Enter query row: "))
```

query_glass = int(input("Enter query glass: "))

print("Output: {:.5f}".format(champagneTower(poured, query_row, query_glass)))

[SAMPLE OUTPUT 1,2,3]